X-CAVATE CTR

Algorithm Reference: Kinematics, Collision Detection, Pathfinding, and Workspace Optimization

Detailed Algorithm Specification

Structure, Runtime Complexity, and Implementation

Date: March 12, 2026
Author: Soham S.
Repository: `CTR-X-CAVATE-GIMBAL`

This document provides a comprehensive reference for every algorithm implemented in the CTR extension of the X-CAVATE vascular network printing platform, with formal runtime analysis, data structure specifications, and implementation details.

Contents

1 System Context: How Algorithms Relate to CTR X-CAVATE **2**

2 Algorithm 1: Rodrigues Rotation **2**
- 2.1 Purpose . . . 2
- 2.2 Mathematical Formulation . . . 3
- 2.3 Usage in the CTR System . . . 3
- 2.4 Runtime and Implementation . . . 3

3 Algorithm 2: Inverse Kinematics (Global → SNT) **3**
- 3.1 Purpose . . . 3
- 3.2 Algorithm . . . 3
  - 3.2.1 Stage 1: Global to Local Transform . . . 3
  - 3.2.2 Stage 2: Local to SNT (Actuator Values) . . . 4
- 3.3 Implementations . . . 4
- 3.4 Default Calibration . . . 4
- 3.5 Implementation Details . . . 4

4 Algorithm 3: Forward Kinematics (Needle Body Computation) **5**
- 4.1 Purpose . . . 5
- 4.2 Algorithm . . . 5
  - 4.2.1 Section 1: Straight SS Tube . . . 5
  - 4.2.2 Section 2: Pre-Curved Nitinol Arc . . . 5
- 4.3 Vectorized Variant (Conflict Graph) . . . 5
- 4.4 Runtime . . . 5
- 4.5 Implementation . . . 6

5 Algorithm 4: CTR Collision Detection **6**
- 5.1 Purpose . . . 6
- 5.2 Data Structures . . . 6
- 5.3 Algorithm: `is_valid(node)` . . . 7
- 5.4 Collision Rules . . . 7
- 5.5 KD-Tree Maintenance . . . 7
- 5.6 Runtime Analysis . . . 8
- 5.7 Implementation . . . 8

6 Algorithm 5: Gimbal Solver with Config Tracking **8**
- 6.1 Purpose . . . 8
- 6.2 Gimbal Configuration Space . . . 8
- 6.3 Algorithm: `GimbalSolverDetector.is_valid(node)` . . . 9
- 6.4 Config Selection Strategy . . . 9
- 6.5 Runtime Analysis . . . 9
- 6.6 Implementation . . . 9

7 Algorithm 6: Swept-Volume Conflict Graph **9**
- 7.1 Purpose . . . 9
- 7.2 Algorithm . . . 10
- 7.3 Runtime Analysis . . . 10
- 7.4 Implementation . . . 10

8 Algorithm 7: Eades GreedyFAS (Cycle Breaking) **10**

8.1 Purpose . . . 10
8.2 Theoretical Guarantee . . . 11
8.3 Algorithm . . . 11
8.4 Runtime Analysis . . . 11
8.5 Application in X-CAVATE . . . 11
8.6 Implementation . . . 11
8.7 Reference . . . 11

9 Algorithm 8: Angular Sector Partitioned DFS 12
9.1 Purpose . . . 12
9.2 Algorithm . . . 12
9.3 DFS Variant: `iterative_dfs_gimbal()` . . . 12
9.4 Runtime Analysis . . . 12
9.5 Implementation . . . 13

10 Algorithm 9: Auto-Placement Optimization 13
10.1 Purpose . . . 13
10.2 Algorithm . . . 13
10.3 Search Space . . . 14
10.4 Runtime Analysis . . . 14
10.5 Design Rationale . . . 14
10.6 Implementation . . . 14

11 Algorithm 10: Gimbal Reachability Expansion 14
11.1 Purpose . . . 14
11.2 Algorithm . . . 14
11.3 Runtime . . . 15
11.4 Impact . . . 15

12 Algorithm 11: Oracle Detector (Diagnostic) 15
12.1 Purpose . . . 15
12.2 Variants . . . 15
12.3 Implementation . . . 15

13 Algorithm 12: Iterative DFS with Collision Checking 15
13.1 Purpose . . . 15
13.2 Variants . . . 15
13.3 Core DFS Loop . . . 16
13.4 Runtime . . . 16
13.5 Implementation . . . 16

14 Summary: Complete Runtime Table 16

15 Data Structure Reference 17

16 Conclusion 17

1 System Context: How Algorithms Relate to CTR X-CAVATE

The X-CAVATE platform converts a 3D vascular network graph into G-code instructions for a bioprinter. The CTR (Concentric Tube Robot) extension replaces the straight-needle Cartesian printer with a two-tube flexible mechanism that reaches target nodes via tube translation and rotation.

This introduces algorithmic challenges absent from straight-needle printing:

1. **Inverse Kinematics (IK):** Given a target point in 3D space, compute the actuator values $(z_{\text{ss}}, z_{\text{ntnl}}, \theta)$ that position the needle tip at that point.
2. **Forward Kinematics (FK):** Given actuator values, compute the 3D shape of the entire needle body (straight section + curved arc), needed for collision detection.
3. **Collision Detection:** Unlike a straight needle (which only occludes points directly below it), the curved CTR body sweeps through a volume. Both *shaft collisions* (hitting unprinted nodes) and *body-drag collisions* (disturbing already-printed material) must be checked.
4. **Workspace Optimization:** The CTR has limited reach. The base position and insertion direction must be optimized to maximize the reachable fraction of the network.
5. **Gimbal Expansion:** A mechanical gimbal tilts the insertion direction within a cone, expanding the reachable workspace. The pathfinder must try multiple gimbal orientations per node.
6. **Print Ordering:** The order in which nodes are visited determines which body-drag collisions occur. A conflict graph encodes precedence constraints, and cycle-breaking heuristics produce near-optimal orderings.

The algorithms are organized into six modules, each described in its own section below.

Table 1: Algorithm Module Map

Module	File	Core Algorithms
Kinematics	`ctr_kinematics.py`	Rodrigues rotation, IK, FK, batch reachability
Collision Detection	`ctr_collision.py`	KD-tree body queries, shaft/drag checks
Gimbal Solver	`gimbal_solver.py`	Multi-config search, solution recording
Swept Volume Ordering	`swept_volume.py`	Conflict graph construction, Eades GreedyFAS
Base Placement	`ctr_placement.py`	Spherical shell sampling, batch evaluation
Pathfinding	`pathfinding.py`	Iterative DFS variants, angular sector partitioning

2 Algorithm 1: Rodrigues Rotation

2.1 Purpose

Rodrigues' rotation formula is the foundational primitive used throughout the CTR system. It computes a 3×3 rotation matrix $\mathbf{R}$ that rotates any vector by angle θ about a unit axis $\hat{k}$.

2.2 Mathematical Formulation

Given unit axis $\hat{k} = (k_x, k_y, k_z)$ and angle θ:

$$\mathbf{K} = \begin{bmatrix} 0 & -k_z & k_y \\ k_z & 0 & -k_x \\ -k_y & k_x & 0 \end{bmatrix}$$

$$\mathbf{R}(\theta, \hat{k}) = \mathbf{I}_3 + \sin(\theta)\,\mathbf{K} + (1 - \cos\theta)\,\mathbf{K}^2$$

2.3 Usage in the CTR System

Table 2: Rodrigues Rotation Usage Across Modules

Caller	Purpose
`CTRConfig.__post_init__()`	Precompute direction matrix $\mathbf{D}$ for fast IK
`global_to_local()`	Build local coordinate frame from $(\hat{R}, \hat{n}, \theta_{\text{match}})$
`compute_needle_body()`	Compute bend direction for nitinol arc
`make_gimbal_config()`	Tilt $\hat{R}$ by (α, ϕ) for gimbal configs
`compute_gimbal_frame()`	Visualization of tilted vs. untilted direction
`build_conflict_graph()`	Vectorized bend direction for all nodes simultaneously

2.4 Runtime and Implementation

- **Time:** $O(1)$ per rotation (fixed 3×3 matrix operations).
- **Space:** $O(1)$ (one 3×3 matrix).
- **File:** `ctr_kinematics.py`, line 147.
- **Vectorized variant:** In `build_conflict_graph()`, the skew-symmetric matrix $\mathbf{K}$ is constructed once and applied to all M nodes simultaneously via broadcasting: $\hat{d}_i = \hat{n} + \sin(a_i)\mathbf{K}\hat{n} + (1 - \cos a_i)\mathbf{K}^2\hat{n}$, achieving $O(M)$ total for M nodes.

3 Algorithm 2: Inverse Kinematics (Global → SNT)

3.1 Purpose

Given a target point $\mathbf{p} \in \mathbb{R}^3$ in the global (network) frame, compute actuator values $(z_{\text{ss}}, z_{\text{ntnl}}, \theta)$ that position the CTR needle tip at $\mathbf{p}$. Returns `None` if the point is outside the reachable workspace.

3.2 Algorithm

The IK pipeline has two stages:

3.2.1 Stage 1: Global to Local Transform

Transform the target point from the global frame to the CTR local frame using the precomputed direction matrix $\mathbf{D}$:

$$\mathbf{D} = \begin{bmatrix} \hat{R} \\ \hat{n}_{\text{rot}} \\ \hat{t} \end{bmatrix}, \quad \hat{n}_{\text{rot}} = \mathbf{R}(\theta_{\text{match}}, \hat{R})\, \hat{n}, \quad \hat{t} = \hat{R} \times \hat{n}_{\text{rot}}$$

$$\boldsymbol{\delta} = \mathbf{p} - \mathbf{X}, \quad (x_l, y_l, z_l)^T = \mathbf{D}\, \boldsymbol{\delta}$$

where $\mathbf{X}$ is the CTR base position, $\hat{R}$ is the insertion direction, and $\hat{n}$ is the bending-plane normal.

3.2.2 Stage 2: Local to SNT (Actuator Values)

1. **Rotation angle:** $\theta = \arctan 2(z_l, y_l)$
2. **Perpendicular distance:** $r_\perp = \sqrt{y_l^2 + z_l^2}$
3. **Calibration bounds check:** If $r_\perp \notin [x_{\text{val}}[0], x_{\text{val}}[-1]]$, return `None`.
4. **Arc parameter:** $y_r = f_{x_r \to y_r}(r_\perp)$ (interpolation lookup)
5. **Extension difference:** $\Delta = f_{y_r \to \text{ntnl-ss}}(y_r)$ (interpolation lookup)
6. **Arc axial contribution:** $a = r \cdot \sin(y_r/r)$ if $y_r > 0$, else 0
7. **SS extension:** $z_{\text{ss}} = x_l - a$
8. **Nitinol extension:** $z_{\text{ntnl}} = z_{\text{ss}} + \Delta$

3.3 Implementations

Three variants exist, optimized for different call patterns:

Table 3: IK Implementation Variants

Function	Input	Runtime	Notes
`global_to_snt()`	1 point	$O(1)$	Uses `interp1d` objects; general purpose
`fast_global_to_snt()`	1 point	$O(1)$	Uses `np.interp` + precomputed $\mathbf{D}$; 3–5× faster
`batch_local_to_snt()`	N points	$O(N)$	Fully vectorized NumPy; amortizes interp overhead

3.4 Default Calibration

When no calibration file is provided, a default analytical calibration based on circular-arc geometry is generated:

$$x_r = r(1 - \cos\alpha), \quad y_r = r\alpha, \quad \Delta = y_r$$

for $\alpha \in [0, \pi/2]$ sampled at 100 points. This assumes the nitinol tube traces a perfect circular arc with no material nonlinearities.

3.5 Implementation Details

- **File:** `ctr_kinematics.py`, lines 171–285 (scalar), 320–432 (batch).
- **Precomputation:** Direction matrix $\mathbf{D}$ and raw interpolation arrays are cached in `CTRConfig.__post_init__()`, avoiding repeated Rodrigues computation.
- **Fast path:** `fast_global_to_snt()` uses Python `math` module for scalar operations and `np.interp` (which is $\sim 5\times$ faster than calling an `interp1d` object for a single point).

4 Algorithm 3: Forward Kinematics (Needle Body Computation)

4.1 Purpose

Given actuator values $(z_{\text{ss}}, z_{\text{ntnl}}, \theta)$, compute the full 3D centerline of the CTR needle body as a sequence of sample points. This is essential for collision detection — unlike a straight needle, the CTR body traces a curved path through space.

4.2 Algorithm

The needle body consists of two concatenated sections:

4.2.1 Section 1: Straight SS Tube

$$\mathbf{p}_{\text{ss}}(t) = \mathbf{X} + t \cdot \hat{R}, \quad t \in [0, z_{\text{ss}}]$$

Sampled at n_{ss} evenly spaced points. The sample count is adaptive:

$$n_{\text{ss}} = \max\left(2, \left\lceil \frac{z_{\text{ss}}}{r \cdot \pi/(2 \cdot n_{\text{arc}})} \right\rceil + 1\right)$$

This ensures the SS sampling density approximately matches the arc sampling density.

4.2.2 Section 2: Pre-Curved Nitinol Arc

If $z_{\text{ntnl}} > z_{\text{ss}}$ (nitinol extends beyond the SS tube):

1. Compute bend direction via simplified Rodrigues (exploiting $\hat{R} \perp \hat{n}$):
$$\hat{d} = \hat{n}\cos(a) + (\hat{R} \times \hat{n})\sin(a), \quad a = \theta_{\text{match}} + \theta$$
2. Compute maximum arc angle: $\alpha_{\text{max}} = (z_{\text{ntnl}} - z_{\text{ss}})/r$
3. Sample n_{arc} points along the circular arc from the SS tip:
$$\mathbf{p}_{\text{arc}}(\alpha) = \mathbf{X}_{\text{ss_tip}} + r\sin(\alpha)\,\hat{R} + r(1 - \cos\alpha)\,\hat{d}$$
for $\alpha \in \{1, 2, \ldots, n_{\text{arc}}\} \cdot \alpha_{\text{max}}/n_{\text{arc}}$.

4.3 Vectorized Variant (Conflict Graph)

In `build_conflict_graph()`, the FK is computed for all M reachable nodes simultaneously:

- All SS sections: $\mathbf{P}_{\text{ss}} \in \mathbb{R}^{M \times n_{\text{ss}} \times 3}$ via broadcasting.
- All arc sections: $\mathbf{P}_{\text{arc}} \in \mathbb{R}^{M \times n_{\text{arc}} \times 3}$ via broadcasting.
- Total body array: $\mathbf{P}_{\text{all}} \in \mathbb{R}^{M \times (n_{\text{ss}} + n_{\text{arc}}) \times 3}$.

4.4 Runtime

Table 4: FK Runtime Complexity

Variant	Time	Space
Scalar (`compute_needle_body`)	$O(n_{\text{ss}} + n_{\text{arc}})$	$O(n_{\text{ss}} + n_{\text{arc}})$
Vectorized (conflict graph)	$O(M \cdot (n_{\text{ss}} + n_{\text{arc}}))$	$O(M \cdot (n_{\text{ss}} + n_{\text{arc}}))$

With default parameters ($n_{\text{ss}} = 12$, $n_{\text{arc}} = 20$), each body has 32 sample points. For $M = 18{,}681$ nodes, the vectorized body array is $18{,}681 \times 32 \times 3 = 1{,}793{,}376$ float64 values ($\approx 14\,\text{MB}$).

4.5 Implementation

- **Scalar:** `ctr_kinematics.py`, line 476, function `compute_needle_body()`.
- **Vectorized:** `swept_volume.py`, lines 117–165 (inline in `build_conflict_graph()`).
- **Key optimization:** The cross product $\hat{K}_n = \hat{R} \times \hat{n}$ is precomputed in `CTRConfig.__post_init__()` and reused for all FK calls on the same config.

5 Algorithm 4: CTR Collision Detection

5.1 Purpose

Determine whether printing a candidate node v is safe given the current state of visited and unvisited nodes. Unlike straight-needle collision detection (which uses a 2D XY shadow), CTR collision requires checking the full 3D curved needle body against all nearby nodes.

5.2 Data Structures

Table 5: Collision Detector State

Structure	Type	Purpose
`_unvisited`	`Set[int]`	Nodes not yet printed
`_visited`	`Set[int]`	Already-printed nodes
`_unvisited_tree`	`cKDTree`	Spatial index over unvisited node positions
`_visited_tree`	`cKDTree`	Spatial index over visited node positions
`_graph`	`Dict[int, List[int]]`	Adjacency list for neighbor exemption

5.3 Algorithm: `is_valid(node)`

Algorithm 1 CTR Collision Validity Check

Require: node v, unvisited set U, KD-trees T_U, T_V, graph G

if $v \notin U$ **then return false**
end if
$(z_{\text{ss}}, z_{\text{ntnl}}, \theta) \leftarrow \text{FAST_GLOBAL_TO_SNT}(v)$
if SNT out of actuator limits **then return false**
end if
$B \leftarrow \text{COMPUTE_NEEDLE_BODY}(z_{\text{ss}}, z_{\text{ntnl}}, \theta)$ ▷ $|B| = n_{\text{ss}} + n_{\text{arc}}$
$\mathbf{p}_v \leftarrow$ coordinates of v
$B_{\text{far}} \leftarrow \{b \in B : \|b - \mathbf{p}_v\|^2 \geq r_{\text{tip}}^2\}$ ▷ Tip exclusion
$N_v \leftarrow G[v]$ ▷ Graph neighbors of v
for all $b \in B_{\text{far}}$ **do** ▷ **Shaft collision check**
  hits $\leftarrow T_U$.query_ball_point(b, r_{nozzle})
  for all $u \in$ hits **do**
    if $u \neq v$ **and** $u \in U$ **and** $u \notin N_v$ **then return false**
    end if
  end for
end for
for all $b \in B_{\text{far}}$ **do** ▷ **Body-drag check**
  if T_V.query_ball_point$(b, r_{\text{nozzle}}) \neq \emptyset$ **then return false**
  end if
end forreturn true

5.4 Collision Rules

1. **Tip proximity exclusion:** Body points within $3 \times r_{\text{nozzle}}$ of the target node are not checked. This prevents adjacent interpolated nodes (spaced at $\sim r_{\text{nozzle}}$) from falsely blocking the candidate.
2. **Shaft collision (unvisited):** Any unvisited non-neighbor node within r_{nozzle} of a non-tip body point blocks the candidate. The needle shaft would displace that node's gel before it is printed.
3. **Body-drag (visited):** Any visited node within r_{nozzle} of a non-tip body point blocks the candidate. The needle would drag through already-printed gel.
4. **Graph-neighbor exemption:** Direct graph neighbors of the candidate are exempt from shaft checks. These nodes are reachable from the candidate and may be printed in the same DFS pass.

5.5 KD-Tree Maintenance

Both trees are rebuilt periodically (every `rebuild_interval` = 500 mark_visited calls) to keep query performance optimal as nodes move between sets.

```
def _rebuild_trees(self):
    uv_idx = np.array(sorted(self._unvisited), dtype=np.intp)
    self._unvisited_tree = cKDTree(self._points[uv_idx])
    v_idx = np.array(sorted(self._visited), dtype=np.intp)
    self._visited_tree = cKDTree(self._points[v_idx])
```

5.6 Runtime Analysis

Let N = total nodes, B = body sample points per node ($\sim$32), k = average KD-tree query hits.

Table 6: Collision Detection Runtime

Operation	**Cost**
IK (fast scalar)	$O(1)$
FK (body computation)	$O(B)$
Tip exclusion filter	$O(B)$
Shaft check (batch KD-tree query)	$O(B \cdot \log N)$ expected
Body-drag check (batch KD-tree query)	$O(B \cdot \log N)$ expected
KD-tree rebuild (amortized)	$O(N \log N/R)$ per call, where R = rebuild interval
Total per `is_valid()` call	$O(B \cdot (\log N + k))$

5.7 Implementation

- **File:** `ctr_collision.py`, class `CTRCollisionDetector` (242 lines).
- **Key optimization:** Uses `query_ball_point` with `return_length=True` for the body-drag check, avoiding full hit-list construction when only existence matters.
- **Batch mode:** The shaft check uses `query_ball_point(far_body, collision_r)` on the entire body array at once (SciPy internally parallelizes this).

6 Algorithm 5: Gimbal Solver with Config Tracking

6.1 Purpose

Extend the collision detector to try multiple gimbal orientations per node, recording which orientation succeeded so that the G-code writer can emit the correct actuator commands.

6.2 Gimbal Configuration Space

The gimbal tilts the insertion direction $\hat{R}$ by angle α at azimuth ϕ within a cone:

- Cone half-angle: $\alpha_{\max}$ (default 15°)
- Tilt levels: n_{tilt} (default 3) at $\alpha = \{5°, 10°, 15°\}$
- Azimuth samples: n_{az} (default 8) at $\phi = \{0°, 45°, \ldots, 315°\}$
- Total configs: $1 + n_{\text{tilt}} \times n_{\text{az}} = 25$ (including the untilted base)

Each config is a full `CTRConfig` with a tilted $\hat{R}'$ computed via Rodrigues rotation:

$$\hat{k}_{\text{tilt}} = \cos(\phi)\,\hat{n} + \sin(\phi)\,\hat{b}, \quad \hat{R}' = \mathbf{R}(\alpha, \hat{k}_{\text{tilt}})\,\hat{R}$$

where $\hat{b} = \hat{R} \times \hat{n}$.

6.3 Algorithm: `GimbalSolverDetector.is_valid(node)`

Algorithm 2 Gimbal Solver Validity Check

Require: node v, configs $C_0, C_1, \ldots, C_{24}$, active config index a

```
valid_configs ← []
order ← [a] + [0, 1, ..., 24] \ {a}                    ▷ Try active config first
for all i ∈ order do
    (z_ss, z_ntnl, θ) ← FAST_GLOBAL_TO_SNT(v, C_i)
    if unreachable or out of limits then continue
    end if
    B ← COMPUTE_NEEDLE_BODY(z_ss, z_ntnl, θ, C_i)
    if SHAFT_AND_DRAG_OK(v, B) then
        valid_configs.append(i)
        if i = a then break                            ▷ Active config works — short-circuit
        end if
    end if
end for
cache valid_configs for mark_visited() return |valid_configs| > 0
```

6.4 Config Selection Strategy

When `mark_visited(node)` is called:

1. If the active config a is in the valid list, choose it (promotes **gimbal continuity** — minimizes retract/reorient cycles in the G-code).
2. Otherwise, choose the first valid config.
3. Update the active config index to the chosen config.
4. Store a `GimbalNodeSolution` recording $(i, \alpha_i, \phi_i, z_{\text{ss}}, z_{\text{ntnl}}, \theta)$.

6.5 Runtime Analysis

Let C = number of gimbal configs (25), B = body points (32), N = nodes.

- **Best case** (active config works): $O(B \cdot \log N)$ — same as base detector.
- **Worst case** (all configs tried): $O(C \cdot B \cdot \log N) = O(25 \cdot 32 \cdot \log N)$.
- **Empirical:** With angular sector ordering, the active config succeeds $> 85\%$ of the time, so the amortized cost approaches the best case.

6.6 Implementation

- **File:** `gimbal_solver.py`, class `GimbalSolverDetector` (lines 77–253).
- **Config precomputation:** All 25 `CTRConfig` objects are built once in `__init__` via `_build_gimbal_configs()`.
- **Solution storage:** `Dict[int, GimbalNodeSolution]` maps node index to solved values.

7 Algorithm 6: Swept-Volume Conflict Graph

7.1 Purpose

Build a directed graph where edge $i \to j$ means "printing node i requires the needle body to pass through node j's location, so i should be printed *before* j." This graph encodes the precedence constraints that, if respected, eliminate body-drag collisions.

7.2 Algorithm

Algorithm 3 Conflict Graph Construction

Require: points P, graph G, CTR config, nozzle radius r

1: **Step 1:** Batch IK for all N graph nodes $\rightarrow (z_{\text{ss}}, z_{\text{ntnl}}, \theta)_i$ $\triangleright\ O(N)$
2: Filter to M reachable nodes
3: **Step 2:** Vectorized FK for all M nodes $\rightarrow$ body array $\in \mathbb{R}^{M\times B\times 3}$ $\triangleright\ O(MB)$
4: **Step 3:** Compute tip exclusion mask $\in \{0,1\}^{M\times B}$ $\triangleright\ O(MB)$
5: **Step 4:** Build single KD-tree over all MB body points $\triangleright\ O(MB\log(MB))$
6: **Step 5:** For each graph node j, query KD-tree for body points within r $\triangleright\ O(N\log(MB))$
7: **Step 6:** For each hit, map body point $\rightarrow$ source node i; add edge $i \rightarrow j$ if $j \notin G[i]$ $\triangleright\ O(E)$

7.3 Runtime Analysis

Table 7: Conflict Graph Construction Complexity

Step	**Time**	**Space**
Batch IK	$O(N)$	$O(N)$
Vectorized FK	$O(MB)$	$O(MB)$
Tip exclusion	$O(MB)$	$O(MB)$
KD-tree build	$O(MB\log(MB))$	$O(MB)$
KD-tree query (all nodes)	$O(N\log(MB)+H)$	$O(H)$
Graph assembly	$O(H)$	$O(E)$
Total	$O(MB\log(MB)+H)$	$O(MB+E)$

where M = reachable nodes, B = body samples (32), H = total KD-tree hits, E = conflict edges.

Empirical: For $M = 18{,}681$ nodes with $B = 32$: KD-tree over $\sim$ 600K points, built in $\sim$ 0.3s. Batch query with `workers=-1` (parallel) completes in $\sim$ 1.5s. Total conflict graph construction: $<$ 3s.

7.4 Implementation

- **File:** `swept_volume.py`, function `build_conflict_graph()` (lines 33–238).
- **Body point $\rightarrow$ source node mapping:** `src_node = valid_nodes[hit_idx // n_body]`. This integer division maps each flat body-point index back to the node that generated it.
- **Parallel queries:** `cKDTree.query_ball_point(..., workers=-1)` uses all CPU cores.

8 Algorithm 7: Eades GreedyFAS (Cycle Breaking)

8.1 Purpose

The conflict graph typically contains cycles (mutual precedence constraints). To obtain a valid linear print ordering, cycles must be broken by removing a minimum set of back-edges. The Eades GreedyFAS algorithm provides a provably good approximation.

8.2 Theoretical Guarantee

The algorithm removes at least 50% of back-edges in any directed graph, running in $O(m)$ time where m is the edge count. This is the best known polynomial-time approximation for the NP-hard Feedback Arc Set problem.

8.3 Algorithm

Algorithm 4 Eades GreedyFAS Linear Ordering

Require: Directed graph $G = (V, E)$
Initialize output deques $S_L \leftarrow \emptyset$, $S_R \leftarrow \emptyset$
Compute indeg$[v]$, outdeg$[v]$, $\delta[v] = \text{outdeg}[v] - \text{indeg}[v]$ for all v
while $V \neq \emptyset$ **do**
  repeat
    while $\exists$ sink v with outdeg$[v] = 0$ **do**
      Remove v from G; prepend v to S_L
    end while
    while $\exists$ source v with indeg$[v] = 0$ **do**
      Remove v from G; append v to S_R
    end while
  until no sinks or sources found
  if $V \neq \emptyset$ **then**
    $v^* \leftarrow \arg\max_{v \in V} \delta[v]$ ▷ Max outdeg − indeg
    Remove v^* from G; append v^* to S_R
  end if
end while
return $S_L \cdot S_R$ ▷ Concatenation

8.4 Runtime Analysis

- **Time:** $O(n + m)$ where $n = |V|$, $m = |E|$. Each node is processed exactly once; each edge is examined at most twice (once for degree update on each endpoint).
- **Space:** $O(n + m)$ for adjacency lists, degree arrays, and bucket queues.

8.5 Application in X-CAVATE

The Eades ordering is applied *within each angular sector* (see Section 9). For a sector with n_s nodes and m_s intra-sector conflict edges, the cost is $O(n_s + m_s)$.

8.6 Implementation

- **File:** `swept_volume.py`, function `eades_greedy_fas()` (lines 241–366).
- **Bucket queues:** Nodes are organized by $\delta = \text{outdeg} - \text{indeg}$ in a `defaultdict(set)` for $O(1)$ access to the maximum-δ node.
- **Reverse adjacency:** A reverse adjacency list (`rev`) enables efficient predecessor lookup during node removal.

8.7 Reference

Eades, P., Lin, X., and Smyth, W.F. (1993). "A fast and effective heuristic for the feedback arc set problem." *Information Processing Letters*, 47(6):319–323.

9 Algorithm 8: Angular Sector Partitioned DFS

9.1 Purpose

Organize the global print ordering by partitioning nodes into angular sectors based on their rotation angle θ, then applying Eades ordering within each sector. This exploits the observation that the CTR bending plane sweeps through 360°, and nodes at similar θ can be printed with minimal gimbal reorientation.

9.2 Algorithm

Algorithm 5 Angular Sector Partitioned DFS

Require: Graph G, points P, CTR config, n_{sectors}

Step 1: Compute θ_i for all nodes via batch IK

Step 2: Partition nodes into n_{sectors} sectors by θ:

$$\text{sector}(i) = \left\lfloor \frac{\theta_i + \pi}{2\pi / n_{\text{sectors}}} \right\rfloor$$

Step 3: Build global conflict graph C via `build_conflict_graph`

for all sector $s = 0, 1, \ldots, n_{\text{sectors}} - 1$ **do**

  Extract intra-sector conflict subgraph C_s

  $\text{order}_s \leftarrow \textsc{eades_greedy_fas}(C_s)$

  Process nodes in Eades order via `iterative_dfs_gimbal`

end for

Cleanup: Process any remaining unvisited nodes via z-heap DFS

9.3 DFS Variant: `iterative_dfs_gimbal()`

A modified DFS that promotes gimbal config continuity:

1. For the current node, collect unvisited neighbors from the graph.
2. Partition neighbors into two tiers:
   - **Tier 1:** Reachable with the detector's *active* gimbal config (quick IK + bounds check).
   - **Tier 2:** All other unvisited neighbors.
3. Within each tier, sort by `conflict_outdegree[node]` ascending (nodes that block fewer future nodes are visited first).
4. Push Tier 2 first, then Tier 1 onto the stack (LIFO ensures Tier 1 is processed first).

This biases the DFS toward neighbors reachable without changing the gimbal orientation, reducing retract/reorient cycles in the physical mechanism.

9.4 Runtime Analysis

Let N = total graph nodes, S = number of sectors (default 8), M = reachable nodes, E = conflict edges.

Table 8: Angular Sector DFS Runtime

Step	Time	Notes
Batch θ computation	$O(N)$	Vectorized IK
Sector partitioning	$O(N)$	Array bin assignment
Conflict graph	$O(MB\log(MB))$	See Section 7
Per-sector Eades	$\sum_s O(n_s + m_s) = O(N + E)$	
DFS passes	$O(N \cdot C \cdot B \cdot \log N)$	C gimbal configs per validity check
Total	$O(MB\log(MB) + NCB\log N)$	Dominated by DFS validity checks

9.5 Implementation

- **File:** `gimbal_solver.py`, function `_run_angular_sector()` (lines 355–489).
- **Sector assignment:** `k = int(np.clip((theta + pi) / sector_width, 0, n_sectors - 1))`.
- **Cleanup sweep:** Handles nodes with undefined θ (unreachable from base config but reachable from a gimbal config) and any nodes not covered by sector processing.

10 Algorithm 9: Auto-Placement Optimization

10.1 Purpose

Automatically determine the optimal CTR base position $\mathbf{X}$ and insertion direction $\hat{R}$ to maximize the fraction of network nodes reachable by the CTR workspace.

10.2 Algorithm

Algorithm 6 Spherical Shell Placement Optimization

Require: Network points P, CTR parameters $(r, z_{\text{ss,max}}, z_{\text{ntnl,max}})$

$\mathbf{c} \leftarrow \text{mean}(P)$ ▷ Network centroid
$r_{\text{net}} \leftarrow \max_i \|P_i - \mathbf{c}\|$ ▷ Bounding sphere radius
$d_{\text{reach}} \leftarrow z_{\text{ss,max}} + r$ ▷ Axial reach
Compute standoff distances $D = \text{linspace}(s_{\min} \cdot r', s_{\max} \cdot r', n_d)$
best $\leftarrow (-1, \mathbf{0}, \mathbf{0})$ ▷ (count, position, orientation)
for all $d \in D$ **do**
  for all $\theta \in [-70°, +70°]$ sampled at n_{el} points **do**
    for all $\phi \in [0, 360°)$ sampled at n_{az} points **do**
      $\hat{u} \leftarrow (\cos\theta\cos\phi, \cos\theta\sin\phi, \sin\theta)$
      $\mathbf{X} \leftarrow \mathbf{c} + d \cdot \hat{u}$
      $\hat{R} \leftarrow (\mathbf{c} - \mathbf{X})/\|\mathbf{c} - \mathbf{X}\|$ ▷ Point toward centroid
      Build `CTRConfig` with $(\mathbf{X}, \hat{R}, r, z_{\text{ss,max}}, z_{\text{ntnl,max}})$
      count $\leftarrow \sum$ BATCH_IS_REACHABLE(P)
      if count $>$ best.count **then**
        best $\leftarrow$ (count, $\mathbf{X}$, $\hat{R}$)
      end if
    end for
  end for
end for
return best

10.3 Search Space

- **Azimuth:** 12 samples in $[0, 2\pi)$
- **Elevation:** 7 samples in $[-70°, +70°]$ (avoid degenerate pole orientations)
- **Distances:** 4 standoff multipliers in $[1.2 \cdot d'/2, 2.5 \cdot d']$ where $d' = r_{\text{net}} + d_{\text{reach}}/2$
- **Total candidates:** $12 \times 7 \times 4 = 336$

10.4 Runtime Analysis

Let K = number of candidates (336), N = network nodes.

- **Per candidate:** `batch_is_reachable` costs $O(N)$ (vectorized matrix multiply + interp + boolean mask).
- **CTRConfig construction:** $O(1)$ (reuses pre-loaded calibration).
- **Total:** $O(K \cdot N) = O(336 \cdot N)$.

For $N = 18{,}681$: $336 \times 18{,}681 \approx 6.3$M reachability checks, completing in < 2 seconds.

10.5 Design Rationale

- **Why spherical shell?** The CTR has a toroidal-like workspace centered on the insertion axis. The base must be at a "Goldilocks" distance: too close and the workspace doesn't enclose the network; too far and nodes exceed the maximum extension. The shell sampling covers the viable zone.
- **Why $\hat{R}$ toward centroid?** This centers the toroidal workspace on the network, empirically maximizing coverage. Non-centroid-pointing orientations consistently perform worse.
- **Why $\pm 70°$ elevation?** Near-vertical placements ($> 70°$) produce degenerate toroid orientations that cover thin slices of the network.

10.6 Implementation

- **File:** `ctr_placement.py`, function `optimize_ctr_placement()` (174 lines).
- **Calibration sharing:** `_load_calibration()` is called once before the candidate loop; all candidates reuse the same interpolation functions.

11 Algorithm 10: Gimbal Reachability Expansion

11.1 Purpose

Determine whether a node is reachable from *any* gimbal configuration, not just the base. This is used during graph pruning to avoid removing nodes that are unreachable from the base but reachable from a tilted orientation.

11.2 Algorithm

```python
def gimbal_reachable_nodes(points, graph_nodes, base_config,
                           cone_angle_deg, n_tilt, n_azimuth):
    configs = _build_gimbal_configs(base_config, cone_angle_deg,
                                    n_tilt, n_azimuth)
    combined = np.zeros(len(graph_nodes), dtype=bool)
    for cfg in configs:
        mask = batch_is_reachable(points[graph_nodes], cfg)
        combined |= mask
    return combined
```

11.3 Runtime

$O(C \cdot N)$ where $C = 25$ configs and N = number of graph nodes. Each `batch_is_reachable` call is $O(N)$ (vectorized). Total: $25N \approx 467$K operations for 18,681 nodes.

11.4 Impact

Without gimbal reachability expansion: 435/18,681 nodes pruned (2.3%). With gimbal expansion (15° cone): 0 nodes pruned (100% coverage). The gimbal tilts $\hat{R}$ enough to shift the workspace boundary past fringe nodes.

12 Algorithm 11: Oracle Detector (Diagnostic)

12.1 Purpose

Establish a theoretical upper bound on the number of nodes printable per pass by disabling the body-drag check. If the production detector (with body-drag) approaches the oracle's performance, the ordering strategy is working well.

12.2 Variants

Table 9: Detector Hierarchy

Detector	Shaft?	Body-drag?	Gimbal?
`OracleDetector`	Yes	**No**	No
`GimbalOracleDetector`	Yes	**No**	Yes
`GimbalRealisticDetector`	Yes	Yes	Yes
`GimbalSolverDetector`	Yes	Yes	Yes + recording
`CTRCollisionDetector`	Yes	Yes	No

12.3 Implementation

- **File:** `oracle.py`, classes `OracleDetector` (lines 38–98), `GimbalOracleDetector` (lines 191–275), `GimbalRealisticDetector` (lines 278–376).
- All inherit from `CTRCollisionDetector`; the oracle variants simply skip the visited-tree query in `is_valid()`.

13 Algorithm 12: Iterative DFS with Collision Checking

13.1 Purpose

Traverse the network graph depth-first, visiting collision-free nodes and recording the visit order as a single print pass. When the DFS stalls (no valid neighbors), a new pass is started from the next unvisited node.

13.2 Variants

Three DFS variants are implemented:

Table 10: DFS Variants

Function	Neighbor Ordering
`iterative_dfs()`	Reverse sorted by node index (lowest first)
`iterative_dfs_greedy()`	Ascending `conflict_outdegree` (fewest future conflicts first)
`iterative_dfs_gimbal()`	Two-tier: active-config-reachable first, then others; within tiers by conflict outdegree

13.3 Core DFS Loop

Algorithm 7 Iterative DFS with Collision Checking

Require: Graph G, start node s, detector D

```
stack ← [s], pass_list ← []
while stack ≠ ∅ do
    v ← stack.pop()
    if D.is_visited(v) then continue
    end if
    if not D.is_valid(v) then continue
    end if
    D.mark_visited(v)
    pass_list.append(v)
    N_v ← sorted unvisited neighbors of v (ordering depends on variant)
    Push N_v onto stack in reverse order
end while
return pass_list
```

13.4 Runtime

Each node is popped from the stack at most once (visited check), so the DFS itself is $O(N + E_G)$ where E_G = graph edges. However, each `is_valid()` call costs $O(CB \log N)$ (see Section 5), making the per-pass cost $O(|P| \cdot CB \log N)$ where $|P|$ = nodes in the pass.

13.5 Implementation

- **File:** `pathfinding.py`, lines 642–787.
- **Why iterative?** The original recursive DFS hit Python's 1000-frame recursion limit on networks with >1000 nodes. The explicit stack has no depth limit.

14 Summary: Complete Runtime Table

Table 11: Complete Algorithm Runtime Summary

Algorithm		Runtime	Empirical (18K nodes)
Rodrigues Rotation		$O(1)$	$< 1\,\mu$s
Scalar (`fast_global_to_snt`)	IK	$O(1)$	$\sim 5\,\mu$s

Algorithm	Runtime	Empirical (18K nodes)
Batch IK (`batch_local_to_snt`)	$O(N)$	$\sim 2\,\text{ms}$
Scalar FK (`compute_needle_body`)	$O(B)$	$\sim 50\,\mu\text{s}$
Vectorized FK (conflict graph)	$O(MB)$	$\sim 100\,\text{ms}$
Single collision check (`is_valid`)	$O(CB \log N)$	$\sim 200\,\mu\text{s}$ (best), $\sim 5\,\text{ms}$ (worst)
Conflict graph construction	$O(MB \log(MB))$	$\sim 3\,\text{s}$
Eades GreedyFAS	$O(n+m)$	$< 100\,\text{ms}$
Angular sector partitioning	$O(N)$	$< 10\,\text{ms}$
Auto-placement (336 candidates)	$O(K \cdot N)$	$< 2\,\text{s}$
Gimbal reachability (25 configs)	$O(C \cdot N)$	$< 500\,\text{ms}$
Full pipeline (all passes)	$O(NCB \log N)$	~ 30–$60\,\text{s}$

Key: N = nodes, M = reachable nodes, B = body samples (32), C = gimbal configs (25), K = placement candidates (336), E = conflict edges, n, m = subgraph nodes/edges.

15 Data Structure Reference

Table 12: Core Data Structures

Structure	Type	Usage
`CTRConfig`	`dataclass`	All geometric/calibration params; precomputed $\mathbf{D}$, $\hat{K}_n$, raw interp arrays
`GimbalNodeSolution`	`dataclass`	Per-node solution: $(i, \alpha, \phi, z_{\text{ss}}, z_{\text{ntnl}}, \theta)$
`cKDTree`	scipy spatial	$O(\log N)$ radius queries for collision detection
`Graph`	`Dict[int, List[int]]`	Network adjacency list
`conflict_graph`	`Dict[int, Set[int]]`	Directed precedence constraints
Flat body array	`ndarray (M*B, 3)`	All body points for KD-tree indexing

16 Conclusion

The CTR X-CAVATE extension implements 12 distinct algorithms organized into a pipeline that transforms a vascular network graph into collision-free, gimbal-aware print passes:

1. Rodrigues rotation provides the geometric primitive for all frame transforms.
2. Inverse kinematics maps 3D targets to actuator commands with scalar, fast-scalar, and batch variants optimized for different call patterns.
3. Forward kinematics computes the curved needle body, enabling full 3D collision envelopes.
4. KD-tree-accelerated collision detection handles shaft and body-drag checks in $O(B \log N)$ per node.

5. The gimbal solver tries up to 25 orientations per node with continuity-aware config selection.
6. The swept-volume conflict graph encodes "must-print-before" precedence constraints via vectorized body computation and a unified KD-tree over ∼600K body points.
7. Eades GreedyFAS breaks cycles in $O(m)$ with a proven 50% back-edge removal guarantee.
8. Angular sector partitioning exploits the rotational structure of the CTR workspace for efficient ordering.
9. Auto-placement optimization finds the best base position from 336 candidates in < 2 seconds.
10. Gimbal reachability expansion eliminates fringe-node pruning, achieving 100% network coverage.

Together, these algorithms enable the CTR to print all 18,681 nodes of a complex vascular network in 101 collision-free passes with zero pruned nodes and zero body-drag violations.